

Núcleo Lógico Evolutivo v7.0

Memory Evolutive Systems, Teoría de Categorías
y Verificación Formal en Lean 4

*Una arquitectura distribuida para el razonamiento
matemático asistido por inteligencia artificial*

Leonardo Jiménez Martínez

Centro de Biomatemáticas · BIOMAT

leonsinmiedo@gmail.com

Marzo 2025

Resumen

Presentamos el **Núcleo Lógico Evolutivo v7.0** (NLE), un sistema de razonamiento matemático formal que integra cuatro paradigmas heterogéneos en un marco arquitectónicamente coherente:

(i) Memory Evolutive Systems (MES) de Ehresmann como inspiración para la organización de la memoria, (ii) teoría de categorías delgadas como lenguaje formal de la estructura de conocimiento, (iii) verificación en Lean 4 como fuente de verdad matemática, y (iv) aprendizaje por refuerzo con Proximal Policy Optimization (PPO) sobre redes neuronales de grafos (GNN) para la selección adaptativa de estrategias.

La arquitectura distribuye el control en una red de cuatro *reguladores de retroalimentación* (tac, org, str, int) que operan en escalas temporales diferenciadas, junto con una jerarquía de 19 *agentes-join estructurales* (4 pilares fundacionales en L_1 , 14 áreas matemáticas en L_2 , 1 orquestador en L_3) que envuelven joins verificados del grafo. El grafo de habilidades se modela en dos niveles: un *quiver* sintáctico de aristas etiquetadas (dep, an, tr) y la *categoría delgada* inducida por su preorden de alcanzabilidad. La evolución del sistema procede por *extensiones por joins*: cada nueva competencia aparece como cota superior mínima de un patrón existente, certificada por un predicado decidible `is_join`.

El sistema incluye 76 habilidades organizadas en 14 categorías y tres niveles de abstracción, una memoria sembrada con 2371 patrones obtenidos de ProofNet y NuminaMath, y un pipeline Lean-first que garantiza que ninguna afirmación matemática se emite sin verificación formal. Reportamos resultados de entrenamiento con 71 663 ejemplos en dos fases (preentrenamiento supervisado y ajuste PPO con recompensas diferenciales), obteniendo exactitud de validación del 100 % en la Fase 1 y recompensa promedio de 0.578 ± 0.008 en la Fase 2. Este trabajo describe la arquitectura del sistema y sus métricas internas; la evaluación comparativa con sistemas de referencia (miniF2F, ProofNet) se reserva para trabajo experimental dedicado.

Palabras clave: Memory Evolutive Systems, teoría de categorías, verificación formal, Lean 4, aprendizaje por refuerzo, razonamiento matemático, redes neuronales de grafos.

Índice general

1	Introducción	4
1.1	Problema central	4
1.2	Contribuciones	4
1.3	Alcance de este paper	5
1.4	Organización del artículo	5

2	Fundamentos teóricos	5
2.1	Memory Evolutive Systems como inspiración arquitectónica	5
2.2	El Principio de Multiplicidad (versión operacional)	6
2.3	Teoría de categorías como lenguaje unificador: del quiver al preorden inducido . .	6
2.4	Verificación formal en Lean 4	7
2.5	Aprendizaje por refuerzo con PPO	7
3	Arquitectura NLE v7.0	7
3.1	Descripción general	7
3.2	El pipeline Lean-first	8
3.3	El grafo categórico de habilidades	9
3.4	El sistema de monitores de dominio	9
3.5	Agentes-join estructurales: pilares, áreas y orquestador	10
3.5.1	Tres capas estructurales	10
3.5.2	Inventario completo	11
3.5.3	Lectura del envoltorio computacional	12
3.5.4	Relación con los reguladores	12
3.6	Memoria evolutiva: extensión por joins y propiedades	12
3.6.1	Separación entre el objeto matemático y el procedimiento de construcción	12
3.6.2	El procedimiento de extensión: graph extension via join verification	14
3.6.3	Axiomas estructurales del grafo	14
3.6.4	Propiedades de la extensión por joins	15
3.6.5	Similaridad de registros de experiencia	15
4	Formalización matemática	16
4.1	El MDP del NLE	16
4.2	La GNN de selección de estrategias	17
4.3	Inicialización de la memoria: semilla de patrones	18
5	Implementación	18
5.1	Estructura del repositorio	18
5.2	Datasets conectados	19
5.3	Métricas del sistema	19
5.4	Proveedores LLM soportados	19
6	Resultados empíricos	19
6.1	Fase 1: preentrenamiento supervisado	19
6.2	Fase 2: ajuste PPO con verificación Lean real	20
6.3	Análisis de la recompensa diferencial	20
6.4	Cobertura de tests	21
7	Discusión y trabajo futuro	21
7.1	Coherencia teórica del diseño	21
7.2	El papel epistémico de Lean	21
7.3	Limitaciones	21
7.4	Trabajo futuro	22
8	Conclusiones	22

A	Axiomas estructurales: verificación formal en Lean 4	24
B	Algoritmo: SolverCascade	25

1 Introducción

El abismo entre la intuición matemática informal y la verificación formal rigurosa representa uno de los problemas más profundos en la intersección de la filosofía de las matemáticas, la lógica computacional y la inteligencia artificial. Un matemático experto opera fluidamente en ambos mundos: navega la intuición de alto nivel mientras puede descender a demostraciones formales cuando la certeza lo exige. Los asistentes de prueba actuales como Lean 4, Coq o Isabelle exigen al usuario articular cada paso en el lenguaje formal del sistema, creando una barrera de entrada que excluye a la mayoría de la comunidad matemática.

Los modelos de lenguaje de gran escala (LLMs) han mostrado capacidades notables de razonamiento informal, pero carecen de garantías de corrección. Un LLM puede generar una demostración plausible que contenga errores sutiles indistinguibles del texto correcto. Esta tensión entre la fluidez expresiva de los LLMs y la certeza formal de los asistentes de prueba motiva la arquitectura del NLE.

1.1 Problema central

Pregunta de Investigación

¿Es posible construir un sistema de razonamiento matemático que combine la expresividad del lenguaje natural, la robustez de la verificación formal y la adaptabilidad cognitiva, preservando al mismo tiempo la coherencia teórica interna?

1.2 Contribuciones

Las contribuciones de este trabajo son:

1. **Arquitectura NLE:** Sistema integrado en cuatro capas —memoria evolutiva, categoría delgada de habilidades, pipeline Lean-first y GNN+PPO— en un único marco arquitectónicamente coherente.
2. **Inventario de agentes en dos clases disjuntas:** Cuatro *reguladores de retroalimentación* (tac, org, str, int) operando como procesos en escalas temporales diferenciadas, más 19 *agentes-join estructurales* (4 PillarAgents en L_1 , 14 ColimitAgents en L_2 , 1 Orquestador en L_3) que envuelven joins verificados del grafo. Las dos clases satisfacen el Principio de Separación proceso/objeto: los reguladores son procesos, los agentes-join son envoltorios computacionales de objetos categóricos.
3. **Modelo categórico de habilidades en dos niveles:** Un quiver etiquetado de 76 habilidades como capa sintáctica, junto con la categoría delgada inducida por su preorden de alcanzabilidad como capa formal.
4. **Separación proceso/objeto:** Distinción explícita entre el join verificado (objeto categórico, único portador de reclamo matemático) y el regulador de retroalimentación que lo produce (proceso computacional, arquitectura inspirada en MES). Esta distinción evita el error de categoría usual en sistemas inspirados en MES y permite el procedimiento de *Graph Extension via Join Verification* certificado por un predicado decidible.
5. **Pipeline Lean-first:** Mecanismo arquitectónico que garantiza verificación formal antes de toda respuesta matemática.

6. **Resultados empíricos:** Entrenamiento en dos fases sobre 71 663 ejemplos con métricas internas de validación.

1.3 Alcance de este paper

Este trabajo describe la *arquitectura* del NLE v7.0: su pipeline Lean-first, su grafo categórico de habilidades, su sistema de monitores de dominio, su esquema GNN+PPO y la formalización en Lean 4 de las propiedades estructurales del grafo. Reportamos resultados de entrenamiento y métricas internas (Sección 6).

No afirmamos que el NLE represente un avance comparativo sobre sistemas establecidos de razonamiento matemático formal (Draft-Sketch-Prove, Hypertree Proof Search, COPRA, LeanDojo) ni sobre el uso directo de un LLM frontera como prover. Tal evaluación requeriría ejecutar el NLE sobre benchmarks estándar —mínimamente miniF2F, idealmente también ProofNet y LeanCat— y comparar con los resultados publicados por los sistemas de referencia. Este trabajo experimental queda explícitamente fuera del alcance del paper presente y se reservará para una publicación de evaluación dedicada (Sección 7.4).

1.4 Organización del artículo

La Sección 2 desarrolla los fundamentos teóricos. La Sección 3 describe la arquitectura v7.0 en detalle. La Sección 4 presenta las definiciones formales y los resultados teóricos. La Sección 5 describe la implementación. La Sección 6 reporta los resultados empíricos internos. La Sección 7 discute implicaciones y trabajo futuro.

2 Fundamentos teóricos

2.1 Memory Evolutive Systems como inspiración arquitectónica

Los Memory Evolutive Systems (MES) fueron introducidos por Ehresmann y Vanbremeersch [1] como un modelo matemático de sistemas cognitivos complejos. Un MES modela la dinámica de un sistema con memoria mediante una sucesión de categorías que evolucionan en el tiempo a través de complexificaciones, junto con una red de co-reguladores que operan en escalas temporales heterogéneas y garantizan robustez por redundancia.

Tomamos de MES tres ideas que estructuran la arquitectura del NLE: (i) la organización jerárquica de la memoria por niveles de abstracción, (ii) la noción de *patrón* y de *síntesis* como objeto cota superior de un patrón existente, y (iii) la distribución de control entre múltiples reguladores de escala temporal disjunta. Estas ideas se traducen al NLE con el siguiente compromiso de fidelidad formal: las construcciones del NLE se enuncian sobre la categoría delgada finita actualmente vigente y son honestas respecto a qué propiedades categóricas se demuestran y cuáles permanecen como inspiración heurística. La distinción se hace explícita en cada definición.

Definición 2.1: Patrón en una categoría delgada

Sea G una categoría delgada finita. Un **patrón** en G es una subfamilia finita no vacía $P = \{a_1, \dots, a_n\} \subseteq \text{Ob}(G)$. La **cota superior mínima** (join) de P , cuando existe, es el objeto $c_P \in \text{Ob}(G)$ caracterizado por:

1. $a_i \leq_G c_P$ para todo $i \in \{1, \dots, n\}$ (cota superior).

2. Para todo $x \in \text{Ob}(G)$ con $a_i \leq_G x$ para todo i , se tiene $c_P \leq_G x$ (minimalidad).
 Cuando existe, $c_P = \bigvee P$ es el colímite del patrón en G y es único.

2.2 El Principio de Multiplicidad (versión operacional)

Principio 2.1: Multiplicidad

El NLE implementa al menos dos monitores de dominio operando concurrentemente con escalas temporales τ_i disjuntas, satisfaciendo

$$\tau_{\text{tac}} \ll \tau_{\text{org}} \ll \tau_{\text{str}} \ll \tau_{\text{int}}.$$

Esta separación garantiza dos propiedades operacionales:

- **Robustez ante fallo:** la interrupción de un monitor no bloquea el flujo principal, dado que las decisiones de menor escala temporal se procesan con monitores rápidos independientes.
- **Cobertura de horizontes:** decisiones inmediatas (clasificación), tácticas (encadenamiento), estratégicas (planificación) e integrativas (consolidación) son atendidas por monitores diseñados para sus escalas respectivas.

La formulación categórica del Principio de Multiplicidad de Ehresmann y Vanbremeersch [1] cuantifica sobre objetos categóricos CR_i y sus ámbitos temporales mediante funtores de binding. La versión operacional aquí adoptada captura las consecuencias arquitectónicas observables (separación de escalas y robustez por redundancia) sin invocar la maquinaria categórica que el NLE no formaliza explícitamente para sus monitores.

2.3 Teoría de categorías como lenguaje unificador: del quiver al preorden inducido

El grafo de habilidades del NLE se especifica en dos niveles que distinguimos explícitamente para evitar la ambigüedad usual entre “grafo dirigido” y “categoría”:

Nivel sintáctico: el quiver etiquetado. Sea $Q = (V, E, s, t, \ell)$ un quiver finito etiquetado donde V es el conjunto de skills, E el conjunto de aristas, $s, t : E \rightarrow V$ las funciones source y target, y $\ell : E \rightarrow \{\text{dep}, \text{an}, \text{tr}\}$ asigna a cada arista uno de tres tipos semánticos:

- **dep** (dependencia): dominar $t(e)$ presupone dominar $s(e)$.
- **an** (analogía): las técnicas de $s(e)$ se transfieren parcialmente a $t(e)$.
- **tr** (traducción): $s(e)$ admite formalización mediante las herramientas de $t(e)$.

Las etiquetas son metadatos semánticos sobre las aristas, no eligen morfismos en una categoría enriquecida. El sistema usa estas etiquetas para razonamiento heurístico (rutas de explicación, sugerencia de prerrequisitos), pero no induce composición tipada.

Nivel categórico: el preorden inducido. Sea G la categoría delgada cuyos objetos son V y cuyo único morfismo $s_i \rightarrow s_j$ existe si y solo si s_j es alcanzable desde s_i por una ruta dirigida en

Q , sin importar la etiqueta de las aristas. Equivalentemente, G es el preorden de alcanzabilidad de Q :

$$s_i \leq_G s_j \iff \exists (e_1, \dots, e_k) \in E^* \text{ con } s(e_1) = s_i, t(e_k) = s_j, t(e_p) = s(e_{p+1}).$$

La estructura categórica que el sistema verifica formalmente —composición asociativa, identidades, existencia de joins— es la del preorden G , no la del quiver Q . Las propiedades demostradas en Lean 4 (Apéndice A) se enuncian sobre G . La distinción quiver/preorden es estándar [11, 10]: el quiver Q admite una categoría libre $\text{Free}(Q)$ cuyos morfismos son rutas formales, pero $\text{Free}(Q)$ no es lo que el sistema implementa; el sistema implementa el cociente de $\text{Free}(Q)$ que identifica todas las rutas con los mismos extremos, y este cociente es G .

2.4 Verificación formal en Lean 4

Lean 4 [3] es un asistente de prueba basado en la Teoría de Tipos Dependientes (en particular, el Cálculo de Construcciones Inductivas Coherentes). En el NLE, Lean 4 cumple un rol epistémico preciso: es la fuente de verdad del sistema.

Toolchain de reproducción. Lean 4.29.0-rc4; Mathlib4 commit fijado en el archivo `lean-toolchain` del proyecto; comando de invocación `lake env lean --json`.

El *sorry cascade* implementa auto-completado de brechas en pruebas mediante tácticas ordenadas por poder expresivo creciente:

$$\text{rfl} \Rightarrow \text{simp} \Rightarrow \text{ring} \Rightarrow \text{omega} \Rightarrow \text{aesop}.$$

2.5 Aprendizaje por refuerzo con PPO

Proximal Policy Optimization [2] es un algoritmo de RL que optimiza una política π_θ mediante la maximización de un objetivo recortado que evita actualizaciones excesivas:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

donde $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ es la razón de probabilidades y $\hat{A}_t$ es la estimación de ventaja.

3 Arquitectura NLE v7.0

3.1 Descripción general

El NLE v7.0 se organiza en cuatro capas jerárquicas, cada una responsable de un nivel de abstracción distinto:

Capa 4 — Interfaz LLM: Formaliza $\leftrightarrow$ Traduce
Capa 3 — Pipeline Lean-first: $\text{rfl} \Rightarrow \text{simp} \Rightarrow \text{ring} \Rightarrow \text{omega} \Rightarrow \text{aesop}$
Capa 2 — Reguladores transversales: $\text{tac} \mid \text{org} \mid \text{str} \mid \text{int}$ (procesos)
Capa 1c — Orquestador L_3: 1 agente-join (punto de entrada)
Capa 1b — ColimitAgents L_2: 14 agentes-join por área matemática
Capa 1a — PillarAgents L_1: 4 agentes-join fundacionales (ZFC, CatThy, Logic, TypeThy)
Capa 1 — Quiver Q + categoría G + Memoria: 76 skills · 14 categorías · 2371 patrones

La Capa 2 contiene *procesos* (reguladores con ciclo de retroalimentación), mientras que las Capas 1a–1c contienen *objetos del grafo* envueltos como agentes-join. La distinción es la del Principio 3.1.

El estado del sistema en el tiempo t se describe formalmente como sigue.

Definición 3.1: Estado del sistema NLE

El estado del NLE en el instante t es la cuádrupla

$$\Sigma_t = (G_t, \mathcal{M}_t, \mathcal{R}_t, \iota_{t,t'})$$

donde:

- G_t es la categoría delgada de habilidades vigente en el instante t (Sección 2.3).
- $\mathcal{M}_t$ es la memoria de patrones: conjunto finito de pares (P_k, c_{P_k}) donde $P_k \subseteq \text{Ob}(G_t)$ y $c_{P_k} = \bigvee P_k$ es su join en G_t .
- $\mathcal{R}_t = \{R_{\text{tac}}, R_{\text{org}}, R_{\text{str}}, R_{\text{int}}\}$ es el conjunto de monitores de dominio.
- $\iota_{t,t'} : G_t \hookrightarrow G_{t'}$ es la inclusión inducida por la extensión por joins (Definición 3.3) ocurrida en el período $[t, t']$.

3.2 El pipeline Lean-first

El principio de diseño más importante del NLE es que **toda consulta matemática pasa obligatoriamente por Lean 4 antes de producir una respuesta**. Esto garantiza que el sistema nunca emite afirmaciones matemáticas no verificadas. El LLM tiene dos roles estrictamente delimitados y complementarios:

1. **Formalizador:** transforma el enunciado en lenguaje natural a código Lean 4.
2. **Traductor:** convierte el resultado verificado de Lean 4 a lenguaje natural accesible.

```

1 import Mathlib.Tactic.NormNum
2
3 -- Generado por el NLE: "¿Es 2 + 2 = 4 un hecho verificable?"
4 theorem nle_verificacion_basica : 2 + 2 = 4 := by norm_num
5
6 -- SolverCascade para una identidad algebraica
7 theorem nle_identidad_algebra (x y : Real) :
8   (x + y)^2 = x^2 + 2*x*y + y^2 := by ring
9

```



```

10 -- Ejemplo con omega para aritmetica lineal
11 theorem nle_divisibilidad (n : Nat) (h : n % 2 = 0) :
12   (n + 2) % 2 = 0 := by omega

```

Listing 1: Ejemplo de formalización y verificación automática en Lean 4. El NLE genera este código a partir de un enunciado en lenguaje natural.

3.3 El grafo categórico de habilidades

Definición 3.2: Categoría de habilidades

El grafo de habilidades del NLE es el par (Q, G) donde:

- $Q = (V, E, s, t, \ell)$ es el **quiver de habilidades** con $|V| = 76$ skills, aristas etiquetadas en $\{\text{dep}, \text{an}, \text{tr}\}$ y estructura jerárquica $\text{lvl} : V \rightarrow \{L_0, L_1, L_2\}$.
- G es la **categoría delgada inducida** por Q : $\text{Ob}(G) = V$, $\text{Hom}_G(s_i, s_j) = \{\bullet\}$ si s_j es alcanzable desde s_i en Q , y $\emptyset$ en caso contrario. La composición y las identidades son las únicas posibles en una categoría delgada (asociatividad y unitalidad triviales).

Toda afirmación posterior sobre “morfismos del grafo de skills” se entenderá como afirmación sobre morfismos en G . Toda afirmación sobre tipos de relación (dep, an, tr) refiere a etiquetas de aristas en Q .

La organización jerárquica de los 76 skills en 14 categorías y tres niveles se muestra en la Tabla 1.

Cuadro 1: Organización del grafo de habilidades en tres niveles y 14 categorías.

Nivel	Categoría	Skills	Ejemplos
L_0	ZFC / Fundamentos	2	Axioma de elección, regularidad
	Teoría de categorías	2	Funtores, transformaciones nat.
	Lógica de primer orden	2	Resolución, secuentes
	Teoría de tipos	2	CIC, tipos dependientes
	Lean 4 (básico)	2	Tácticas, Mathlib
L_1	Álgebra	8	Grupos, anillos, módulos
	Análisis	8	Continuidad, medida, EDO
	Geometría	6	Diferencial, algebraica
	Topología	6	Homotopía, cohomología
	Combinatoria	6	Grafos, generadores
	Probabilidad	6	Estocástico, martingalas
	Computación	6	Complejidad, semántica
	Optimización	6	Convexa, entera, control
	Cat. avanzadas	8	Topos, ∞ -cat., adjunciones
L_2	Estrategias de prueba	6	Inducción, reducción al absurdo

3.4 El sistema de monitores de dominio

El NLE implementa el Principio de Multiplicidad (Principio 2.1) mediante una red de cuatro **reguladores de retroalimentación** (*feedback regulators*) distribuidos, cada uno responsable de su propia escala temporal y dominio de decisión. Cuando se enfatiza su localización arquitectónica, los denominamos **monitores de dominio**; cuando se enfatiza su función como procedimientos

que observan el grafo, proponen modificaciones y verifican condiciones antes de aceptarlas, los denominamos *feedback regulators*. Ambos términos refieren a las mismas cuatro entidades R_{tac} , R_{org} , R_{str} , R_{int} .

Cada regulador instancia un ciclo de retroalimentación de cuatro fases: (i) construcción de paisaje (observación del estado del sistema), (ii) selección de objetivos, (iii) codificación de procedimiento, (iv) evaluación posterior. Conforme al Principio 3.1 (separación proceso/objeto), los reguladores son procesos de construcción y verificación, no objetos categóricos: producen joins (a través del procedimiento de la Sección 3.6.2) pero no son ellos mismos joins ni colímites.

Cuadro 2: Los cuatro monitores de dominio del NLE v7.0 y sus propiedades.

Monitor	Nombre	Escala	Rol	Tráf.
R_{tac}	Tactical	milisegundos	Clasifica consultas y enruta al pipeline	80 %
R_{org}	Organizational	segundos	Organiza secuencias de tácticas	—
R_{str}	Strategic	minutos	Estrategia de alto nivel	20 %
R_{int}	Integrative	horas	Integra resultados parciales	—

La separación de escalas temporales es fundamental: mientras R_{tac} toma decisiones rápidas de clasificación, R_{str} puede deliberar sobre estrategias a largo plazo sin bloquear el flujo de procesamiento. La inspiración para esta arquitectura proviene de la red de co-reguladores de los Memory Evolutive Systems de Ehresmann y Vanbremeersch [1], donde múltiples co-reguladores con escalas temporales heterogéneas garantizan robustez cognitiva. La propiedad operacional que sí se satisface en el NLE es la separación de escalas (Tabla 2), que es la propiedad arquitectónica que el principio busca capturar.

Frecuencia de operación. Cada regulador opera con una frecuencia distinta sobre el flujo de interacciones: R_{tac} se ejecuta en cada interacción (clasifica response vs assist); R_{org} cada 10 pasos (detecta patrones sin join, depura skills débiles); R_{str} cada 100 pasos (detecta enlaces complejos, propone extensiones); R_{int} cada 50 pasos (verifica invariantes, detecta fracturas, resuelve conflictos). Estas frecuencias materializan la separación temporal $\tau_{tac} \ll \tau_{org} \ll \tau_{str} \ll \tau_{int}$ del Principio 2.1.

3.5 Agentes-join estructurales: pilares, áreas y orquestador

Además de los cuatro reguladores transversales (Sección 3.4), el NLE implementa una segunda clase de entidades —los **agentes-join estructurales**— que no son procesos de regulación sino *envoltorios computacionales de joins verificados*. Su función es proveer puntos de entrada y composición para el grafo G , no observar ni modificar el sistema.

Conforme al Principio 3.1 (separación proceso/objeto), estos agentes-join son objetos del grafo en su rol matemático y, al mismo tiempo, envoltorios de software en su rol computacional. El paper aclara aquí que el reclamo categórico recae exclusivamente sobre el join verificado por `is_join`, no sobre el envoltorio: el envoltorio puede ofrecer interfaces, tácticas Lean por defecto y métodos de invocación, pero estos no son atributos categóricos del join.

3.5.1 Tres capas estructurales

Los agentes-join estructurales se organizan en tres capas que reflejan los tres niveles del quiver Q :

Capa L_1 — Cuatro *PillarAgents* (fundacionales). Cada *PillarAgent* envuelve el join verificado de los skills L_0 de un pilar fundacional. La Tabla 3 resume su estructura.

Cuadro 3: Los cuatro *PillarAgents* de la capa L_1 . Cada uno envuelve $\bigvee P_i$ donde P_i es el patrón de skills L_0 del pilar correspondiente, verificado por `is_join`.

PillarAgent	Skills L_0 que integra	Áreas L_1 que nutre
join[ZFC]	conjuntos, axiomas, ordinales, cardinales	álgebra, teoría de conjuntos, combinatoria, teoría de números, probabilidad, análisis
join[CatThy]	funttores, transformaciones nat., límites, adjunciones	teoría de categorías, topología, álgebra, análisis
join[Logic]	FOL, lógica proposicional, semántica	lógica, estrategias de prueba, tácticas Lean, computación, teoría de conjuntos
join[TypeThy]	STLC, System-F, tipos dependientes	tácticas Lean, estrategias de prueba, computación, lógica

Capa L_2 — Catorce *ColimitAgents* (uno por área matemática). Cada *ColimitAgent* envuelve el join verificado de los skills de su área. Las áreas son: álgebra, análisis, teoría de categorías, combinatoria, computación, geometría, tácticas Lean, lógica, teoría de números, optimización, probabilidad, estrategias de prueba, teoría de conjuntos, topología. Cada agente-join L_2 porta una táctica Lean por defecto seleccionada según el dominio (por ejemplo, `ring` para álgebra, `norm_num` para teoría de números, `linarith` para optimización lineal). La táctica por defecto es un atributo del envoltorio computacional, no del join en sí.

Capa L_3 — Un *Orquestador*. El orquestador envuelve el join verificado del patrón formado por los catorce *ColimitAgents* de L_2 . Es el punto de entrada único del sistema: toda consulta matemática inicia su recorrido en él y desciende por la jerarquía hasta los pilares según las dependencias detectadas.

3.5.2 Inventario completo

La Tabla 4 resume los $4 + 14 + 1 = 19$ agentes-join estructurales más los 4 reguladores transversales, totalizando 23 entidades activas en el sistema.

Cuadro 4: Inventario completo de agentes en el NLE v7.0. Las dos clases son disjuntas en función pero ambas están sujetas al Principio 3.1: los agentes-join son envoltorios de objetos categóricos, los reguladores son procesos.

Clase	Cantidad	Función	Capa
PillarAgents	4	Envoltorio de join de skills L_0 por pilar fundacional	L_1
ColimitAgents	14	Envoltorio de join de skills por área matemática	L_2
Orquestador	1	Envoltorio de join de los 14 ColimitAgents	L_3
Subtotal agentes-join	19	Objetos categóricos envueltos en software	
Reguladores ($R_{tac}, R_{org}, R_{str}, R_{int}$)	4	Procesos de retroalimentación con ciclo de 4 fases	transversal
Total entidades activas	23		

3.5.3 Lectura del envoltorio computacional

Una observación importante para la lectura de la implementación: los archivos de código `pillar_agents.py` y `colimit_agents.py` contienen comentarios y docstrings que describen a cada agente como “el colímite de” su patrón. Bajo el Principio 3.1, la lectura precisa es la siguiente:

Cada `PillarAgent` y cada `ColimitAgent` es un envoltorio computacional sobre el join verificado por `is_join` del patrón correspondiente en G_n . El reclamo matemático es sobre el join (objeto del grafo, verificable decidiblemente); el envoltorio aporta la interfaz Python, la táctica Lean por defecto y los métodos de invocación, pero estos atributos no son parte del reclamo categórico.

La estructura de datos es la misma de antes; lo que cambia es el reclamo verbal en la documentación: el agente *envuelve* el join, no *es* el join. Esta lectura preserva la coherencia entre el código y la formalización en `JoinColimit.lean` (Apéndice A).

3.5.4 Relación con los reguladores

Las dos clases de agentes interactúan así:

- Los **reguladores** ($R_{\text{tac}}, R_{\text{org}}, R_{\text{str}}, R_{\text{int}}$) observan G_n , detectan patrones sin join existente, y disparan el procedimiento de *Graph Extension via Join Verification* (Sección 3.6.2) cuando es apropiado.
- Los **agentes-join estructurales** (L_1, L_2, L_3) son los nodos persistentes del grafo: ya existían antes de la observación del regulador o fueron creados como resultado de una extensión exitosa.

En particular: cuando R_{str} propone una extensión y `is_join` la certifica, el nuevo nodo c_P se materializa como un nuevo agente-join en la capa correspondiente (típicamente L_2). El regulador es el productor; el agente-join es el producto. Esta es la articulación operacional del Principio 3.1.

3.6 Memoria evolutiva: extensión por joins y propiedades

La memoria del NLE evoluciona mediante *extensiones por joins*. Esta operación es nuestra contraparte computable de la complexificación de Ehresmann y Vanbremeersch [1]: se distingue de ella en un aspecto crucial, ya que la complexificación original se define por una propiedad universal sobre todos los funtores hacia categorías arbitrarias, mientras que la extensión por joins es una operación combinatoria sobre la categoría delgada finita actual.

3.6.1 Separación entre el objeto matemático y el procedimiento de construcción

Antes de definir la extensión por joins, el NLE adopta una distinción metodológica que estructura toda esta sección y elimina una fuente común de confusión categórica:

Principio 3.1: Separación proceso/objeto

En el NLE se distinguen dos roles que la literatura informal sobre MES suele unificar bajo el término “agente”:

- **El objeto matemático.** El *join* c_P de un patrón P en G_n . Es un nodo del grafo, estático, caracterizado por dos condiciones decidibles —ser cota superior y ser minimal entre las cotas— que se verifican computacionalmente con un predicado `is_join` y se especifican formalmente en Lean 4 (Apéndice A). Es el único elemento de la arquitectura con reclamo categórico: en una categoría delgada finita, $\text{join} = \text{colímite}$.
- **El procedimiento de construcción.** El *regulador de retroalimentación* (*feedback regulator*) que observa G_n , detecta patrones sin join existente, propone un candidato y verifica que satisface las dos condiciones; si las satisface, el candidato entra al grafo como join certificado. Es proceso, no objeto. Su semántica es computacional —arquitectura inspirada en MES— y no porta carga categórica adicional más allá de la que hereda de los objetos sobre los que opera.

Confundir ambos roles bajo un solo nombre llevaría a un error de categoría: identificar el constructor con la construcción, o el verificador con lo verificado. El paper reserva el término **join** para el objeto y los términos **regulador de retroalimentación** y **monitor de dominio** para el proceso. Los monitores de dominio del NLE (Sección 3.4, donde se les renombra “*feedback regulators*” cuando se enfatiza este aspecto) son procedimientos, no objetos categóricos.

Bajo esta convención, la extensión por joins puede definirse limpiamente como una operación sobre G_n cuyo resultado es un nuevo grafo G_{n+m} en el que ciertos patrones que carecían de join ahora lo tienen. El procedimiento que decide cuándo y cómo se aplica esta operación es separado y se describe en la Sección 3.6.2.

Definición 3.3: Extensión por joins

Sea G_n una categoría delgada finita y sea $\mathcal{P} = \{P_1, \dots, P_k\}$ una familia finita de patrones admisibles en G_n (Definición 2.1). La **extensión de G_n por $\mathcal{P}$** , denotada $G_n[\mathcal{P}]$, es la categoría delgada G_{n+m} obtenida mediante:

1. Para cada $P_i \in \mathcal{P}$ que no admita join en G_n (i. e., no existe $j \in \text{Ob}(G_n)$ que sea cota superior mínima de P_i), se añade un objeto fresco c_{P_i} con $\text{lvl}(c_{P_i}) = 1 + \max_{a \in P_i} \text{lvl}(a)$.
2. Se añaden las aristas $a \rightarrow c_{P_i}$ para cada $a \in P_i$ (estructura de cocono).
3. Se completa la relación de alcanzabilidad por clausura transitiva: $c_{P_i} \rightarrow x$ se añade si y solo si todo $a \in P_i$ ya tenía $a \rightarrow x$ en G_n . Es decir, c_{P_i} hereda exclusivamente las cotas superiores comunes de P_i .

El objeto c_{P_i} es el **join sintetizado** del patrón P_i en G_{n+m} .

En una categoría delgada, los colímites finitos son joins del preorden subyacente: $\text{colim}(P) = \bigvee_{a \in P} a$ cuando este existe. La extensión por joins satisface la propiedad universal del colímite dentro de G_{n+m} , por construcción de los pasos 2 y 3. Lo que no satisface, y por tanto no se afirma, es la universalidad respecto a funtores hacia categorías arbitrarias D , que es la propiedad de la complexificación de Ehresmann.

3.6.2 El procedimiento de extensión: graph extension via join verification

La extensión por joins (Definición 3.3) es una operación matemática sobre grafos. El *procedimiento* que decide cuándo aplicarla, qué patrones son candidatos y verifica que el resultado satisface las condiciones de join se denomina **Graph Extension via Join Verification**. Es el proceso ejecutado por los reguladores de retroalimentación del NLE (típicamente R_{str} y R_{int} , en escalas de minutos a horas), y su descripción es la siguiente:

1. **Observación.** El regulador escanea G_n y la memoria de patrones $\mathcal{M}_t$ para identificar patrones $P \subseteq \text{Ob}(G_n)$ activos en consultas recientes que carezcan de join existente.
2. **Propuesta.** Para cada patrón candidato P , se construye un nodo provisional c_P junto con la estructura de cocono.
3. **Verificación.** El predicado decidible $\text{is_join}(c_P, P, G_{n+1})$ certifica las dos condiciones: c_P es cota superior de P y, para toda otra cota superior x , se cumple $c_P \leq_{G_{n+1}} x$. La verificación es decidible porque G_{n+1} es finito.
4. **Aceptación.** Si y solo si la verificación tiene éxito, c_P se incorpora a G_{n+1} como join certificado. En caso de fallo, c_P se descarta y el patrón P permanece sin join.

El predicado is_join y la corrección del procedimiento están especificados formalmente en el archivo `JoinColimit.lean` del repositorio (Apéndice A). El nombre *Graph Extension via Join Verification* comunica con precisión qué hace el procedimiento: extiende el grafo en aquellos puntos donde la verificación de join tiene éxito, y solo en esos. La distinción terminológica es importante: el procedimiento no *construye* el colímite en sentido categórico (el colímite ya estaba determinado por P y el preorden); el procedimiento *verifica* que un candidato satisface las condiciones que lo califican como join, y al hacerlo pasa el grafo de un estado a otro.

Reclamo matemático y reclamo computacional. La distinción del Principio 3.1 se materializa aquí: el único reclamo matemático es sobre el join c_P una vez verificado por is_join ; el procedimiento que lo produjo es arquitectura computacional inspirada en MES, no MES formal. Los reguladores de retroalimentación, los criterios de selección de patrones y la política de aceptación son decisiones de diseño que pueden cambiar sin alterar la corrección del join verificado.

3.6.3 Axiomas estructurales del grafo

La memoria evolutiva del NLE satisface cuatro axiomas estructurales que se demuestran en Lean 4 (Apéndice A).

Principio 3.2: Jerarquía

Para todo patrón $P \subseteq \text{Ob}(G_t)$ con join c_P , el nivel del join es estrictamente mayor que el de cualquier componente:

$$\forall a \in P : \text{lvl}(a) < \text{lvl}(c_P).$$

Principio 3.3: Multiplicidad estructural

Todo objeto $c \in \text{Ob}(G_t)$ que sea join puede ser join de al menos dos patrones distintos P_1, P_2 con $P_1 \neq P_2$, garantizando robustez ante fallos de pattern-matching individuales.

Principio 3.4: Conectividad

Para todo par $s_i, s_j \in \text{Ob}(G_t)$, existe una cadena de morfismos en G_t de la forma $s_i = a_0 \rightarrow a_1 \rightarrow \dots \rightarrow a_k = s_j$, posiblemente pasando por nodos intermedios.

Principio 3.5: Cobertura de pilares

Todo objeto de G_t es alcanzable desde alguno de los pilares de L_0 mediante morfismos del preorden.

3.6.4 Propiedades de la extensión por joins**Teorema 3.1: Crecimiento por extensión, propiedades relativas**

Sea G_n una categoría delgada de habilidades en el instante t que satisface los Principios 3.2–3.5. Sea $\mathcal{P}$ un conjunto de patrones admisibles en G_n y sea $G_{n+m} = G_n[\mathcal{P}]$ la extensión por joins (Definición 3.3). Entonces:

1. **Crecimiento estricto:** $|\text{Ob}(G_{n+m})| > |\text{Ob}(G_n)|$, salvo que todos los patrones de $\mathcal{P}$ ya tuvieran join en G_n .
2. **Persistencia de los axiomas:** G_{n+m} satisface los Principios 3.2–3.5.
3. **Inclusión plena:** El funtor canónico $\iota : G_n \hookrightarrow G_{n+m}$ es fiel y pleno: para todo $a, b \in \text{Ob}(G_n)$, $\text{Hom}_{G_n}(a, b) \cong \text{Hom}_{G_{n+m}}(a, b)$.
4. **Relatividad de los joins:** Para todo objeto $c \in \text{Ob}(G_n)$ que sea join de un patrón P en G_n , c continúa siendo join de P en G_{n+m} *si y solo si* ningún objeto añadido $c_{P'}$ con $P' \cap \uparrow P \neq \emptyset$ resulta intercalado entre P y c en el preorden extendido. En general, esta condición debe re-verificarse al añadir nuevos skills.

Esbozo. Los puntos (1)–(3) son consecuencias inmediatas de la construcción de la Definición 3.3: ι no añade ni identifica morfismos sobre $\text{Ob}(G_n)$, y los axiomas se preservan por inspección sobre los nuevos nodos. (4) se sigue de la observación de que un join es la menor cota superior, y “menor” depende de qué objetos hay disponibles como cotas; la adición de un objeto puede crear una cota intermedia que invalide la minimalidad previa. $\square$

Convención sobre afirmaciones de join. Toda afirmación de la forma “ c es el join de $\{a_1, \dots, a_k\}$ ” en este paper se entenderá como afirmación **relativa a G_n en el instante t** , dentro de la categoría delgada actualmente vigente. Tras una extensión $G_n \mapsto G_{n+m}$, los joins previamente verificados deben re-evaluarse según el Teorema 3.1 (4).

3.6.5 Similitud de registros de experiencia

La memoria del NLE almacena, además de los pares patrón–join, registros de experiencia $r = (\text{pid}, v)$ que asocian un identificador de patrón pid con un valor de éxito $v \in \mathbb{R}$ obtenido en evaluaciones previas. Para agrupar registros con desempeño funcional indistinguible, se introduce la siguiente relación.

Definición 3.4: ε -similaridad de registros

Dada una memoria de experiencia $\mathcal{E} = \{r_1, r_2, \dots\}$ con $r_i = (\text{pid}_i, v_i)$, y un parámetro $\varepsilon > 0$, la ε -**similaridad** es la relación binaria $\sim_\varepsilon \subseteq \mathcal{E} \times \mathcal{E}$ definida por:

$$r_i \sim_\varepsilon r_j \iff \text{pid}_i = \text{pid}_j \wedge |v_i - v_j| < \varepsilon \wedge \text{sgn}(v_i) = \text{sgn}(v_j).$$

En la implementación actual, $\varepsilon = 0.1$.

Proposición 3.1: Naturaleza de $\sim_\varepsilon$

La relación $\sim_\varepsilon$ es *reflexiva* (trivialmente $r \sim_\varepsilon r$) y *simétrica* (la definición es invariante bajo intercambio). **No es transitiva:** pueden existir r_1, r_2, r_3 con $r_1 \sim_\varepsilon r_2$ y $r_2 \sim_\varepsilon r_3$ pero $r_1 \not\sim_\varepsilon r_3$ (cuando $|v_1 - v_3|$ alcanza valores hasta 2ε). Por tanto, $\sim_\varepsilon$ es una **relación de tolerancia** [13], no una relación de equivalencia.

Para usos donde se requiera una partición (no una mera tolerancia), el NLE considera la *clausura transitiva* $\approx_\varepsilon = (\sim_\varepsilon)^*$: la relación de conexión por componentes en el grafo cuyas aristas son los pares ε -similares. Esta sí es una relación de equivalencia, pero pierde la garantía local de proximidad —dos registros $\approx_\varepsilon$ -equivalentes pueden estar arbitrariamente lejos en valor.

4 Formalización matemática

4.1 El MDP del NLE

El proceso de decisión de Markov subyacente al aprendizaje del NLE se define como sigue.

Definición 4.1: MDP del NLE

El MDP del NLE es la quintupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: espacio de estados, donde cada $s = (q, G_t, m_t) \in \mathcal{S}$ codifica la consulta q , el grafo de habilidades actual G_t y el estado de memoria $m_t \in \mathcal{M}_t$.
- $\mathcal{A} = \{\text{responder}, \text{asistir_lean}\}$: las dos acciones posibles del **agente PPO de routing**, que opera dentro del regulador táctico R_{tac} . Conforme al Principio 3.1, el término “agente” aquí refiere exclusivamente al decisor del MDP en sentido de aprendizaje por refuerzo —la política π_θ realizada por la SkillGNN+actor-crítico— y no a un objeto categórico ni a un regulador en sí mismo.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$: función de transición.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: función de recompensa.
- $\gamma \in (0, 1)$: factor de descuento.

La función de recompensa diferencial implementada en el NLE es:

$$\mathcal{R}(s, a) = \begin{cases} +1.0 & \text{asistir_lean} + \boxed{N} + \text{norm_num} \checkmark \\ +0.5 & \text{asistir_lean} + \text{problema abstracto} \\ +0.3 & \text{asistir_lean} + \text{timeout/error Lean} \\ -0.5 & \text{responder para consulta matemática} \end{cases}$$

La recompensa diferencial es crucial: el agente aprende a discriminar entre tipos de problemas y a seleccionar la estrategia óptima para cada caso.

4.2 La GNN de selección de estrategias

Definición 4.2: SkillGNN

La **SkillGNN** es una red neuronal de grafos con arquitectura de atención multi-cabeza:

$$h_v^{(l+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} W^{(l)} h_u^{(l)} \right)$$

donde $\alpha_{vu}^{(l)}$ son los coeficientes de atención del l -ésimo Graph Attention Layer (GATConv) y $W^{(l)}$ es la matriz de pesos aprendibles. La arquitectura consta de tres capas GATConv con cuatro cabezas de atención y dimensión 256, seguidas de una proyección a 128 dimensiones.

Ámbito. La SkillGNN está entrenada para *routing binario* (asistir_lean vs responder), no para selección de tácticas dentro del pipeline de Lean.

El desglose de parámetros se muestra en la Tabla 5.

Cuadro 5: Desglose de parámetros entrenables del NLE. Los valores por capa son aproximados; el total es el valor reportado por `model.parameters()`.

Módulo	Parámetros (aprox.)	Notas
node_proj ($d_{\text{feat}} \times 64$)	$\approx 1,024$	proyección de entrada
GATConv-1 ($64 \rightarrow 256$, 4 heads)	$\approx 66,560$	atención multi-cabeza
GATConv-2 ($256 \rightarrow 256$, 4 heads)	$\approx 131,072$	atención multi-cabeza
GATConv-3 ($256 \rightarrow 256$, 4 heads)	$\approx 131,072$	atención multi-cabeza
out_proj ($256 \rightarrow 128$)	$\approx 32,896$	proyección de salida GNN
shared_net ($384 \rightarrow 256 \rightarrow 128$)	$\approx 148,736$	capas compartidas AC
actor ($128 \rightarrow 3$)	387	política
critic ($128 \rightarrow 1$)	129	estimador de valor
Total (verificado)	546,820	≈ 2.2 MB

Justificación arquitectónica de la SkillGNN. La elección de tres capas GATConv con 546 820 parámetros es una decisión de diseño orientada a la extensibilidad futura del sistema, no una superioridad empírica demostrada sobre alternativas más simples para la tarea actual. Tres consideraciones la sustentan:

1. *Tarea actual vs. tarea diseñada.* El routing binario actual (asistir_lean vs responder) es separable con clasificadores paramétricamente mucho menores —esperaríamos que una regresión logística o un MLP de dos capas alcanzasen exactitud cercana al 100 % sobre los splits actuales. La SkillGNN se diseñó para que la misma arquitectura escale a tareas más finas (selección entre múltiples skills, generación de orden de aplicación) cuando el grafo crezca y la decisión deje de ser binaria.
2. *Atención sobre estructura de grafo.* Las capas GATConv con atención multi-cabeza permiten que la representación de un skill incorpore información de su vecindad en el preorden G . Para un grafo de 76 nodos, este beneficio es marginal sobre la tarea actual; para grafos

extendidos por joins sucesivos (Definición 3.3), la representación contextualizada es la motivación arquitectónica.

3. *Ablación pendiente.* Reconocemos como limitación que este paper no reporta una comparación cuantitativa entre la SkillGNN y baselines paramétricamente menores (regresión logística, MLP, k -NN sobre embeddings de query). Tal estudio queda como trabajo experimental pendiente (Sección 7.4).

No afirmamos en este paper que la SkillGNN sea *necesaria* o *superior* para la tarea de routing actual; afirmamos que es una elección arquitectónica orientada a futura extensibilidad cuya validación empírica está pendiente.

4.3 Inicialización de la memoria: semilla de patrones

La memoria $\mathcal{M}_0$ se inicializa con 2371 patrones procedentes de dos fuentes:

- **ProofNet** [8]: 371 teoremas formales verificados, que proveen patrones con estructura lógica precisa.
- **NuminaMath**: 2000 problemas con solución, que proveen patrones de reconocimiento y resolución.

Cada patrón P_k se almacena como un par (P_k, c_{P_k}) donde c_{P_k} es el join computado en la categoría de habilidades —la habilidad emergente que sintetiza los skills involucrados en resolver P_k .

5 Implementación

5.1 Estructura del repositorio

El NLE v7.0 está implementado en Python 3.10 con aproximadamente 12 800 líneas de código organizadas en módulos especializados:

```

1  metamath-prover/
2  +-- app.py                # Interfaz Streamlit (chat)
3  +-- pages/
4  |   +-- 1_Visualizaciones.py
5  +-- nucleo/               # ~12,800 LOC
6  |   +-- core.py           # Orquestador (pipeline Lean-first)
7  |   +-- graph/            # Quiver Q + categoría G
8  |   +-- mes/              # Memoria + extensión por joins
9  |   +-- rl/               # GNN + PPO
10 |   +-- lean/             # Cliente Lean 4 + SolverCascade
11 |   +-- llm/              # Cliente LLM multi-proveedor
12 |   +-- pillars/          # Fundamentos: ZFC, FOL, tipos
13 +-- MetamathProver/       # Pruebas Lean 4 (8 archivos)
14 +-- scripts/              # seed, benchmark, train
15 +-- tests/                # 379 tests en 17 suites

```

Listing 2: Estructura del repositorio del NLE v7.0.

Cuadro 6: Datasets conectados al NLE v7.0 y su rol en el sistema.

Dataset	Rol	Cantidad
ProofNet [8]	Memoria: teoremas formales verificados	371
miniF2F [7]	Few-shot Lean: ejemplos inyectados en prompt	157
NuminaMath	Memoria: reconocimiento de patrones	2000
MATH [9]	Benchmark: evaluación de respuestas	ref.
GSM8K	Benchmark: razonamiento numérico	ref.
Total (entrenamiento)	Fases 1 y 2 combinadas	71 663

5.2 Datasets conectados

5.3 Métricas del sistema

El NLE distingue dos clases de métricas para separar la contribución del núcleo de la contribución del LLM externo:

Cuadro 7: Métricas del NLE v7.0.

Métrica	Tipo	Descripción
Exactitud de respuesta	LLM-dep.	Tolerancia 10^{-4} , comparación simbólica
Matching <code>\boxed{}</code>	LLM-dep.	Extrae y compara respuesta final
skill_hit	NLE esp.	Skill correcto identificado en el grafo
cr_source	NLE esp.	R_{tac} enrutó correctamente
lean_verified	NLE esp.	Lean verificó sin <code>sorry</code>
memory_hit	NLE esp.	Memoria recuperó patrón previo

5.4 Proveedores LLM soportados

El NLE es agnóstico respecto al proveedor LLM mediante una capa de abstracción que soporta: Google AI Studio (Gemini 2.0 Flash, 1.5 Pro), Groq (Llama 3.3 70B, Mixtral 8x7B), Anthropic (Claude Haiku, Sonnet) y un modo Demo sin API externa.

6 Resultados empíricos

6.1 Fase 1: preentrenamiento supervisado

El preentrenamiento supervisado entrena al agente para seleccionar `asistir_lean` en todo problema matemático, estableciendo el comportamiento base del pipeline Lean-first. Los 71 663 ejemplos se particionan en tres splits antes del entrenamiento:

Cuadro 8: Splits del conjunto de entrenamiento (Fase 1).

Split	Ejemplos	Proporción
Entrenamiento	52 237	72.9 %
Validación	6 552	9.1 %
Prueba	12 874	18.0 %
Total	71 663	100 %

Cuadro 9: Resultados del preentrenamiento supervisado (Fase 1).

Época	Train Acc.	Val. Acc.	Test Acc.
1	100 %	100 %	100 %

La convergencia en la primera época indica que la tarea de clasificación binaria (asistir vs responder) es perfectamente separable con la representación provista por la SkillGNN. Esto valida que el grafo categórico de habilidades codifica una señal discriminativa suficiente, aunque, como se discute en la Sección 4.2, no permite concluir superioridad de la GNN sobre clasificadores más simples sin un estudio de ablación dedicado.

6.2 Fase 2: ajuste PPO con verificación Lean real

La Fase 2 refina el modelo usando recompensas diferenciales generadas por la ejecución real de Lean 4 con Mathlib.

Cuadro 10: Resultados detallados de la Fase 2 (PPO).

Época	Loss	Avg Reward	ASSIST %
1	0.0746	0.573	100 %
2	0.0421	0.568	100 %
3	0.0163	0.562	100 %
4	0.0198	0.570	100 %
5	0.0172	0.578	100 %

El 100 % de assist durante toda la Fase 2 confirma que el agente ha aprendido el comportamiento deseado: nunca intenta responder matemáticas directamente desde el LLM, siempre delega a Lean 4.

6.3 Análisis de la recompensa diferencial

La recompensa promedio $\bar{r} \approx 0.57$ es explicable analíticamente. Sea $p_1 = 0.70$ la proporción de problemas con `\boxed{N}` verificables y $p_2 = 0.30$ la de problemas abstractos. Entonces:

$$\bar{r} = p_1 \cdot r_1 + p_2 \cdot r_2 = 0.70 \cdot 1.0 + 0.30 \cdot 0.5 = 0.85.$$

El valor observado ≈ 0.57 es menor porque un porcentaje de los problemas verificables resultan en timeout o error de Lean (recompensa +0.3). Si $p_3 \approx 0.40$ de los problemas aritméticos fallan en Lean:

$$\bar{r} = 0.70 \cdot (0.60 \cdot 1.0 + 0.40 \cdot 0.3) + 0.30 \cdot 0.5 = 0.70 \cdot 0.72 + 0.15 = 0.654.$$

El valor observado 0.578 es consistente con una tasa de fallo en Lean de aproximadamente 55 %, lo que refleja la dificultad inherente de la formalización automática de problemas arbitrarios de NuminaMath.

Cuadro 11: Resumen de la suite de tests del NLE v7.0 (379 tests en 17 suites).

Suite	Tests	Comp.	Suite	Tests	Comp.
Joins y patrones	28	Memoria	GNN / SkillGNN	41	GNN
Extensión por joins	22	Memoria	PPO / Agente	38	RL
Emergencia	18	Memoria	Lean Client	35	Lean
Multiplicidad	15	Memoria	SolverCascade	28	Táct.
Monitores de dominio	32	Mon.	Memoria	30	Mem.
Aprendizaje vivo	22	RL	CLI / Integración	20	Sist.
Total: 379 tests · Pilares y benchmark: 50 · 17 suites					

6.4 Cobertura de tests

7 Discusión y trabajo futuro

7.1 Coherencia teórica del diseño

El NLE satisface el Principio de Multiplicidad operacional (Principio 2.1) mediante la distribución del control en cuatro monitores con escalas temporales diferenciadas. El esquema GNN+PPO opera dentro del monitor táctico R_{tac} , encapsulado en su dominio de decisión. La coordinación entre monitores emerge de su interacción distribuida, no de un árbitro centralizado.

Proposición 7.1: Complejidad de entrenamiento

La arquitectura distribuida del NLE tiene complejidad de entrenamiento $\mathcal{O}(|\mathcal{A}|^2 \cdot d)$, donde $|\mathcal{A}| = 2$ es el número de acciones y d la dimensión de los embeddings. Esto es tratable en la práctica y explica la convergencia rápida observada en la Fase 1.

7.2 El papel epistémico de Lean

La decisión de hacer de Lean 4 la fuente de verdad tiene consecuencias filosóficas profundas. El sistema no pretende que el LLM sea correcto; más bien, trata al LLM como un traductor de heurística cuya salida siempre debe ser validada por un sistema formal. Esto es análogo al rol del lenguaje informal en la práctica matemática real: la intuición guía, pero la demostración formal decide.

7.3 Limitaciones

1. **Alcance de la GNN.** La SkillGNN está entrenada únicamente para la tarea de routing binario. No realiza selección de tácticas dentro del pipeline de Lean 4; esa responsabilidad recae en la SolverCascade determinista.
2. **Cobertura de Mathlib.** No todo teorema matemático está en Mathlib. Para dominios sin cobertura, el sistema recurre a la recompensa de +0.3.
3. **Formalización automática.** La generación automática de código Lean a partir de lenguaje natural tiene una tasa de éxito limitada, especialmente para enunciados abstractos.
4. **Escalabilidad del grafo.** El grafo de 76 habilidades cubre los dominios más comunes, pero no es exhaustivo.

5. **Re-verificación de joins ante crecimiento.** El sistema actualmente no re-verifica automáticamente las propiedades de join cuando se añaden nuevos skills (Teorema 3.1 (4)). En la práctica, esto significa que las afirmaciones de join son válidas en G_n al instante t y pueden requerir re-evaluación tras una extensión.
6. **Ausencia de comparación externa.** Este paper no reporta resultados sobre miniF2F, ProofNet ni LeanCat, ni comparación con sistemas como Draft-Sketch-Prove, Hypertree Proof Search, COPRA o LeanDojo. La evaluación comparativa es trabajo experimental dedicado.

7.4 Trabajo futuro

Las direcciones de investigación inmediatas incluyen:

1. **Evaluación comparativa en benchmarks estándar.** Ejecutar el NLE sobre miniF2F, ProofNet y LeanCat, y reportar los resultados junto con los baselines de los sistemas de referencia.
2. **Estudio de ablación de la SkillGNN.** Comparar sistemáticamente la GNN con regresión logística, MLP simple y k -NN sobre embeddings de query, sobre los mismos splits de entrenamiento y test.
3. **Re-verificación automática de joins.** Implementar un mecanismo que, ante cada extensión $G_n \mapsto G_{n+m}$, re-verifique las afirmaciones de join afectadas según el Teorema 3.1 (4).
4. **Expansión del grafo.** Agregar habilidades en áreas emergentes (matemáticas computacionales cuánticas, teoría de $(\infty, 1)$ -categorías).
5. **Formalización mejorada.** Fine-tuning del LLM en pares (lenguaje natural, código Lean) para mejorar la tasa de éxito.
6. **Monitores adaptativos.** Permitir que los monitores ajusten dinámicamente sus escalas temporales basados en la carga.
7. **Memoria distribuida.** Federar la memoria entre múltiples instancias del NLE para aprendizaje colectivo.
8. **Interfaz universitaria.** Desplegar el sistema como asistente para cursos de matemáticas en la UNAM, recolectando datos reales de interacción.

8 Conclusiones

Hemos presentado el Núcleo Lógico Evolutivo, un sistema de razonamiento matemático que integra inspiración en Memory Evolutive Systems, teoría de categorías delgadas, verificación en Lean 4 y aprendizaje por refuerzo en un único marco arquitectónicamente coherente.

El sistema se distingue por tres propiedades estructurales:

1. Una red distribuida de monitores de dominio con escalas temporales diferenciadas, satisfaciendo una versión operacional del Principio de Multiplicidad.
2. Un pipeline Lean-first que garantiza verificación formal ante toda respuesta matemática.

3. Un grafo de habilidades modelado en dos niveles —quiver etiquetado y categoría delgada inducida— con evolución por extensión por joins, cuyas propiedades estructurales se demuestran formalmente sobre la categoría vigente G_n .

El NLE demuestra la viabilidad de *construir* sistemas de razonamiento matemático que sean a la vez expresivos en lenguaje natural, rigurosos en verificación formal y adaptativos en aprendizaje. La pregunta de si el NLE supera a sistemas estado-del-arte en benchmarks como miniF2F es una pregunta de evaluación comparativa que requiere los experimentos descritos en la Sección 7.4. La contribución que sí afirmamos en este paper es metodológica: la integración coherente de cuatro paradigmas (MES, categorías delgadas, Lean, GNN+PPO) en un mismo marco, con verificación formal de las propiedades estructurales del grafo de habilidades sobre la categoría delgada vigente.

Agradecimientos

El autor agradece al Centro de Biomatemáticas BIOMAT por el apoyo institucional, y a las comunidades de Lean 4, Mathlib, ProofNet y NuminaMath por los datos y herramientas que hacen posible este trabajo.

Referencias

- [1] Ehresmann, A. C. y Vanbreemersch, J.-P. (2007). *Memory Evolutive Systems: Hierarchy, Emergence, Cognition*. Studies in Multidisciplinarity, vol. 4. Elsevier.
- [2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. y Klimov, O. (2017). Proximal Policy Optimization Algorithms. [arXiv:1707.06347](#).
- [3] de Moura, L. y Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *Proceedings of CADE-28*, LNCS.
- [4] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. y Bengio, Y. (2018). Graph Attention Networks. *Proceedings of ICLR 2018*.
- [5] Fischbach, M. A. y Walsh, C. T. (2024). Problem choice and decision trees in science and engineering. *Cell*, 187, 1828–1833.
- [6] Lawvere, F. W. y Schanuel, S. H. (2009). *Conceptual Mathematics: A First Introduction to Categories*, 2nd ed. Cambridge University Press.
- [7] Zheng, K., Han, J. y Polu, S. (2022). miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. *Proceedings of ICLR 2022*.
- [8] Azerbayev, Z., Paster, K., Chan, C., Santos, B., McAleer, S., Jiang, A., Welleck, S., Kakade, S. y Philipose, M. (2023). ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. [arXiv:2302.12433](#).
- [9] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D. y Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. *Proceedings of NeurIPS 2021*.
- [10] Awodey, S. (2010). *Category Theory*, 2nd ed. Oxford University Press.
- [11] Mac Lane, S. (1998). *Categories for the Working Mathematician*, 2nd ed. Graduate Texts in Mathematics, vol. 5. Springer.

- [12] Miller, D. (2025). *Proof Theory and Logic Programming: Computation as Proof Search*. Cambridge University Press.
- [13] Zeeman, E. C. (1962). The topology of the brain and visual perception. En M. K. Fort (ed.), *Topology of 3-Manifolds and Related Topics*, pp. 240–256. Prentice-Hall.

A Axiomas estructurales: verificación formal en Lean 4

A continuación se presenta la formalización en Lean 4 del predicado decidible `is_join` y de los axiomas estructurales del grafo de habilidades del NLE. Estos archivos están incluidos en el directorio `MetamathProver/` del repositorio.

Predicado de join y verificación (`JoinColimit.lean`)

El reclamo categórico del NLE se concentra en el predicado `is_join`, que decide si un objeto candidato es el join de un patrón en una categoría delgada finita.

```

1  import Mathlib.CategoryTheory.Category.Basic
2  import Mathlib.Order.Bounds.Basic
3
4  -- En una categoria delgada (preorden), un join se especifica
5  -- por dos condiciones decidibles sobre un grafo finito G_n.
6  def is_join {V : Type*} [DecidableEq V] (le : V -> V -> Prop)
7    [DecidableRel le] (P : Finset V) (c : V) : Prop :=
8    -- (1) c es cota superior de P
9    (forall a in P, le a c) /\
10   -- (2) c es minimal entre las cotas superiores de P
11   (forall x : V, (forall a in P, le a x) -> le c x)
12
13  -- En una categoria delgada finita, join = colimite.
14  theorem join_eq_colimit_in_thin {V : Type*} [Category V] [Thin V] [Finite
15    V]
16    (P : Finset V) (c : V) (h : is_join (· <= ·) P c) :
17    IsColimit (Cocone.mk c (cocone_from_upper_bound P c h.1)) := by
18    -- Demostracion por la propiedad universal del minimo en preorden
19    -- finito.
20    sorry

```

Listing 3: Predicado `is_join`: especificación del único reclamo matemático del sistema.

Procedimiento de extensión. El predicado `is_join` sirve como certificación del procedimiento descrito en la Sección 3.6.2: un candidato c_P se acepta en G_{n+1} si y solo si `is_join` retorna verdadero. La decidibilidad se sigue de la finitud de G_{n+1} y de la decidibilidad del preorden $\leq_G$.

Axiomas estructurales del grafo

```

1  import Mathlib.CategoryTheory.Category.Basic
2  import Mathlib.CategoryTheory.Limits.HasLimits
3
4  -- Axioma de Jerarquia: el join tiene nivel mayor que sus componentes
5  axiom mes_hierarchy (K : Type*) [Category K]

```



```

6  (level : K -> Nat) (P : Fin n -> K) (cP : K)
7  (h_colim : IsColimit (Coccone.mk cP _)) :
8  forall i : Fin n, level (P i) < level cP
9
10 -- Axioma de Multiplicidad estructural: existencia de al menos dos
    patrones
11 axiom mes_multiplicity (K : Type*) [Category K] (s : K) :
12   exists P1 P2 : Fin 2 -> K,
13     IsColimit (Coccone.mk s _) /\
14     IsColimit (Coccone.mk s _) /\
15     P1 != P2
16
17 -- Cobertura de Pilares: toda habilidad alcanzable desde LO
18 axiom mes_pillar_coverage (K : Type*) [Category K]
19   (pillars : Finset K) (s : K) :
20   exists p in pillars, Nonempty (p --> s)

```

Listing 4: Formalización de los axiomas estructurales en Lean 4.

B Algoritmo: SolverCascade

Algoritmo 1 SolverCascade: Resolución automática de brechas en pruebas

Entrada: Enunciado Lean φ con `sorry`

Salida: Prueba verificada o fallo con diagnóstico

1: $T \leftarrow [\text{rfl}, \text{simp}, \text{ring}, \text{omega}, \text{aesop}]$

2: **for** $\tau \in T$ **do**

3: $\varphi' \leftarrow \varphi[\text{sorry} \mapsto \tau]$

4: $r \leftarrow \text{LeanVerify}(\varphi')$

5: **if** $r = \text{OK}$ **then**

6: **return** φ'

▷ Prueba completada

7: **end if**

8: **end for**

9: **return** $\langle \text{FALLO}, \text{sorry permanece} \rangle$
